

Výstupy ŠVP**INF-14** Navrhnu a vytvořím strukturu vzájemného propojení tabulek; navrhnu procesy zpracování dat. (*Informatika 6, 3/2*)**Učivo:** vývoj informačního systému – návrh struktury a propojení více tabulek, cizí klíč, relace

Lehké úlohy procvičí vztahy a cizí klíče na databázi kina, střední spojují tabulky dotazem a navrhnou strukturu, těžké řeší vazební tabulky, mazání, pořadí importu a omezení. Pojmy a vzorce máš v listu Must know. Vyber si, kolik úloh spočítáš; řešení si ukládej do svého portfolia.

Lehké ★☆☆

- Urči typ vztahu (1 : 1, 1 : N, M : N): film – jeho promítání; sál – promítání; herec – film; promítání – prodaná vstupenka; občan – jeho platný občanský průkaz; praktický lékař – registrovaný pacient; student vysoké školy – předmět, který si zapsal.
- Do které tabulky patří cizí klíč a jak se bude jmenovat? (a) film – promítání; (b) sál – promítání; (c) promítání – vstupenka; (d) herec – film.
- Databáze kina (tabulky použiješ i v dalších úlohách). (a) Který záznam porušuje referenční integritu? (b) Kolikrát se promítá film Robot Emil? (c) Které filmy se hrají v Malém sále? (d) Který film nemá žádné promítání?

```

film
id |  nazev          |  delka
1  |  Robot Emil     |   95
2  |  Poslední vlak  |  118
3  |  Tajemství hradu |  102
4  |  Zimní příběh   |   88

sál
id |  nazev  | kapacita
1  |  Velký  |   200
2  |  Malý   |    60

promitani
id | film_id | sál_id | cas
10 | 1       | 1      | 2027-03-12 17:00
11 | 2       | 1      | 2027-03-12 20:00
12 | 1       | 2      | 2027-03-12 18:30
13 | 3       | 2      | 2027-03-12 20:30
14 | 2       | 1      | 2027-03-13 19:00
15 | 5       | 2      | 2027-03-13 17:00

```

- Napiš příkaz CREATE TABLE pro tabulku promitani s cizími klíči na tabulky film a sál. Proč jsou oba cizí klíče NOT NULL?

```

CREATE TABLE promitani (
  id INTEGER PRIMARY KEY,
  film_id INTEGER NOT NULL REFERENCES film(id),
  sál_id INTEGER NOT NULL REFERENCES sál(id),
  cas TEXT NOT NULL
);

```

- Film Robot Emil hrají herci s id 1 a 2, Poslední vlak herci 2, 3 a 4, Tajemství hradu herci 1, 3 a 5 a Zimní příběh herec 4. Kolik řádků má vazební tabulka obsazení? Kolik herců hraje ve více než jednom filmu?
- Které z příkazů databáze kina (se zapnutou kontrolou cizích klíčů) odmítne a proč? Příkazy se provádějí postupně.

```

PRAGMA foreign_keys = ON;
INSERT INTO promitani VALUES (16, 4, 2, '2027-03-14 16:00'); -- a)
INSERT INTO promitani VALUES (17, 2, 3, '2027-03-14 18:00'); -- b)
INSERT INTO vstupenka VALUES (9, 16, 'A1', 120);             -- c)
INSERT INTO vstupenka VALUES (10, 10, 'A5', 150);             -- d)
INSERT INTO vstupenka VALUES (11, 99, 'B1', 150);             -- e)

```

7. Napiš dotaz, který vypíše název filmu a čas všech promítání ve Velkém sále, seřazené podle času. Jaký je výsledek?

```
SELECT f.nazev, p.cas
FROM promitani p
JOIN film f ON p.film_id = f.id
JOIN sal s ON p.sal_id = s.id
WHERE s.nazev = 'Velký'
ORDER BY p.cas;
```

8. Autoškola eviduje žáky, instruktory, auta a jízdy (kdo jel, s kým, v jakém autě, kdy a kolik minut). Navrhni tabulky s primárními a cizími klíči. Jaký vztah je mezi žákem a instruktorem a jak je v návrhu zachycen?
9. Kino má jedinou plochou tabulku prodej (id, film, delka_filmu, sal, kapacita_salu, cas, sedadlo, cena). Popiš na příkladu tři problémy (anomálie): při vložení, při změně a při smazání dat.
10. Zaměstnanec kina má nadřízeného, který je také zaměstnanec. Cizí klíč proto odkazuje do stejné tabulky. Co vrátí dotaz? Kolik podřízených má vedoucí Král? Proč má ředitelka nadřizeny_id NULL?

```
CREATE TABLE zamestnanec (
    id INTEGER PRIMARY KEY,
    jmeno TEXT NOT NULL,
    nadrizeny_id INTEGER REFERENCES zamestnanec(id)
);
INSERT INTO zamestnanec VALUES (1, 'Ředitelka Nová', NULL), (2, 'Vedoucí Král', 1),
(3, 'Pokladní Malý', 2), (4, 'Promítač Veselý', 2), (5, 'Účetní Černá', 1);

SELECT z.jmeno, n.jmeno AS nadrizeny
FROM zamestnanec z
LEFT JOIN zamestnanec n ON z.nadrizeny_id = n.id
ORDER BY z.id;
```

11. Navrhni proces „prodej vstupenky“ nad databází kina. U každého kroku uveď, které tabulky se čtou a do kterých se zapisuje. Jak databáze zabrání tomu, aby se stejné sedadlo prodalo dvakrát?
12. Kolik vstupenek se prodalo na film Poslední vlak a jaká je celková tržba? Napiš dotaz přes tři tabulky a urči výsledek. (Tabulka vstupenka: id, promitani_id, sedadlo, cena – promítání 10: 3 vstupenky po 150, 150 a 120 Kč; 11: 2 po 180 Kč; 12: 1 za 150 Kč; 14: 2 po 180 Kč.)

```
SELECT COUNT(*), SUM(v.cena)
FROM vstupenka v
JOIN promitani p ON v.promitani_id = p.id
JOIN film f ON p.film_id = f.id
WHERE f.nazev = 'Poslední vlak';
```

Těžké ★★☆☆

13. Které filmy nemají žádné promítání? Vysvětli, proč dotaz používá LEFT JOIN a podmínku IS NULL. Co by vrátil stejný dotaz s obyčejným JOIN?

```
SELECT f.nazev
FROM film f
LEFT JOIN promitani p ON p.film_id = f.id
WHERE p.id IS NULL;
```

14. Ve kterých filmech hraje Petr Kos a kdo s ním v nich hraje? Popiš, jak dotaz „prochází“ vazební tabulku dvakrát, a urči výsledek.

```
SELECT f.nazev, h2.jmeno
FROM herec h
JOIN obsazeni o ON o.herec_id = h.id
JOIN film f ON f.id = o.film_id
JOIN obsazeni o2 ON o2.film_id = f.id
JOIN herec h2 ON h2.id = o2.herec_id
WHERE h.jmeno = 'Petr Kos' AND h2.id <> h.id
ORDER BY f.nazev, h2.jmeno;
```

15. Odhad velikosti: Kino má 4 sály s kapacitou 200, 120, 80 a 60 míst, v každém je denně 5 promítání a průměrná obsazenost je 40 %. Kolik řádků přibude za rok (365 dní) do tabulky promítání a kolik do tabulky vstupenka? Kolik řádků by měla tabulka film, když kino uvede 250 filmů za rok? Co z toho plyne pro indexy?
16. Co program vypíše? Co se stalo s promítáními filmu 1? Co by se změnilo, kdyby cizí klíč měl ON DELETE RESTRICT? Které nastavení je pro kino vhodnější?

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("PRAGMA foreign_keys = ON")
con.executescript("""
CREATE TABLE film (id INTEGER PRIMARY KEY, nazev TEXT);
CREATE TABLE promitani (id INTEGER PRIMARY KEY,
    film_id INTEGER REFERENCES film(id) ON DELETE CASCADE,
    cas TEXT);
INSERT INTO film VALUES (1, 'Robot Emil'), (2, 'Poslední vlak');
INSERT INTO promitani VALUES (10, 1, '17:00'), (11, 2, '20:00'), (12, 1, '18:30');
""")
con.execute("DELETE FROM film WHERE id = 1")
print(con.execute("SELECT COUNT(*) FROM promitani").fetchone()[0])
try:
    con.execute("INSERT INTO promitani VALUES (13, 3, '18:00')")
except sqlite3.IntegrityError as e:
    print("chyba:", e)
```

17. Grafy × databáze: Do prázdné databáze kina se importují data ze šesti souborů (film, sal, herec, promítání, obsazení, vstupenka). Při zapnutých cizích klíčích musí odkazovaný záznam existovat dřív než záznam, který na něj odkazuje. Nakresli graf závislostí (šipka od odkazované tabulky k odkazující) a urči pořadí importu. Je pořadí jediné?
18. Rozvrh: tabulka hodina propojuje třídu, učitele, učebnu a předmět (cizí klíče trida_id, ucitel_id, ucebna_id). Doplň omezení, aby databáze nepřijala (a) dvě hodiny téže třídy současně, (b) učitele ve dvou třídách současně, (c) dvě hodiny v téže učebně současně. Ověř na příkladu: je zapsána hodina třídy 1 s učitelem 7 v učebně 12 v pondělí 3. hodinu; projde zápis třídy 2 s učitelem 8 v učebně 12 ve stejný čas? A třídy 2 s učitelem 7 v učebně 14?

```
CREATE TABLE hodina (
    id INTEGER PRIMARY KEY,
    trida_id INTEGER NOT NULL,
    ucitel_id INTEGER NOT NULL,
    ucebna_id INTEGER NOT NULL,
    predmet TEXT NOT NULL,
    den INTEGER CHECK (den BETWEEN 1 AND 5),
    poradi INTEGER CHECK (poradi BETWEEN 1 AND 10),
    UNIQUE (trida_id, den, poradi),
    UNIQUE (ucitel_id, den, poradi),
    UNIQUE (ucebna_id, den, poradi)
);
```

- 1 : N; 1 : N; M : N; 1 : N; 1 : 1; 1 : N; M : N.
- (a) promítani.film_id; (b) promítani.sal_id; (c) vstupenka.promítani_id; (d) ani do jedné – vztah M : N potřebuje vazební tabulku obsazení (film_id, herec_id, role).
- (a) promítání 15 – film s id 5 neexistuje (v dalších úlohách je smazané); (b) 2× (promítání 10 a 12); (c) Robot Emil, Tajemství hradu; (d) Zimní příběh.
- Promítání bez filmu nebo bez sálu nedává smysl – každé promítání musí mít obojí (ověřeno v SQLite: odkaz na neexistující film databáze se zapnutými cizími klíči odmítne).
- 9 řádků (jeden řádek = jeden herec v jednom filmu); ve více filmech hrají 4 herci (id 1, 2, 3 a 4), herec 5 jen v jednom.
- a) projde (film 4 i sál 2 existují); b) odmítne – sál 3 neexistuje; c) projde – promítání 16 už vzniklo v a); d) odmítne – sedadlo A5 na promítání 10 je už prodané (UNIQUE); e) odmítne – promítání 99 neexistuje.
- Dotaz v poli s kódem; výsledek: Robot Emil 2027-03-12 17:00; Poslední vlak 2027-03-12 20:00; Poslední vlak 2027-03-13 19:00.
- zak(id PK, jmeno, ...), instruktor(id PK, jmeno, ...), auto(id PK, spz UNIQUE, model), jizda(id PK, zak_id FK, instruktor_id FK, auto_id FK, zacatek, minut). Žák–instruktor je M : N (žák jezdí s více instruktory a naopak); vazební tabulkou je jizda, která nese i další údaje.
- vložení: nový film nejde zapsat, dokud se na něj neprodá vstupenka; změna: oprava délky filmu nebo kapacity sálu se musí udělat v mnoha řádcích a snadno se na některý zapomene (nekonistence); smazání: smazáním poslední vstupenky na film zmizí i informace o filmu.
Řešení: rozdělit na tabulky film, sal, promítani, vstupenka.
- Výsledek: Ředitelka Nová – NULL; Vedoucí Král – Ředitelka Nová; Pokladní Malý – Vedoucí Král; Promítač Veselý – Vedoucí Král; Účetní Černá – Ředitelka Nová; Král má 2 podřízené; ředitelka nikoho nad sebou nemá (LEFT JOIN ji proto ve výsledku ponechá).
- 1) zákazník vybere film a čas (čte film, promítání); 2) zobrazí se volná místa (čte sal – kapacita, vstupenka – obsazená sedadla daného promítání); 3) zákazník vybere sedadlo a zaplatí; 4) zápis do vstupenka (INSERT s promítani_id, sedadlem a cenou); 5) potvrzení. Dvojímu prodeji zabrání omezení UNIQUE (promítani_id, sedadlo) – druhý zápis selže, i kdyby dva lidé klikli současně.
- 4 vstupenky, tržba 720 Kč (promítání 11 a 14).
- Výsledek: Zimní příběh. LEFT JOIN ponechá i filmy bez promítání – jejich sloupce z tabulky promítani jsou NULL, a právě ty podmínka vybere. S obyčejným JOIN by film bez promítání ve spojení vůbec nebyl – dotaz by nevrátil nic.
- Od herce přes obsazení k filmům, pak znovu přes obsazení ke všem hercům těch filmů (kromě něj samého). Výsledek: Poslední vlak – Jana Horáková; Poslední vlak – Tomáš Beneš; Robot Emil – Eva Malá.
- promítani: $4 \cdot 5 \cdot 365 = 7\,300$; vstupenka: $(200 + 120 + 80 + 60) \cdot 5 \cdot 0,4 = 920$ denně, tj. 335 800 ročně; film jen 250. Nejvíce se hledá ve vstupenkách podle promítání → index na vstupenka.promítani_id – cizí klíče, podle kterých se spojuje, je vhodné indexovat.
- Výstup: 1 / chyba: FOREIGN KEY constraint failed – smazáním filmu se kaskádově smazala i jeho 2 promítání, zůstalo 1; vložení promítání neexistujícího filmu 3 databáze odmítla. S RESTRICT by smazání filmu selhalo („FOREIGN KEY constraint failed“) a promítání by zůstalo 3. Pro kino je bezpečnější RESTRICT – nesmazat omylem promítání, na která jsou prodané vstupenky.
- Šipky: film → promítani, sal → promítani, promítani → vstupenka, film → obsazení, herec → obsazení. Pořadí (topologické uspořádání): nejdřív film, sal, herec (libovolně mezi sebou), pak promítani a obsazení, nakonec vstupenka. Není jediné – např. obsazení může jít před promítani nebo za ním.
- Tři omezení UNIQUE – viz kód (cizí klíče vynechány kvůli stručnosti); oba zápisy databáze odmítne: první kvůli učebně 12, druhý kvůli učitel 7; projde až třeba zápis ve 4. hodinu (ověřeno v SQLite).